Students Mini-Hackathon

Pre-Hackathon Developer Guide: Building Fintech Solutions on Stellar

1. What is Stellar?

Stellar is an open-source blockchain network designed for fast cross-border payments, asset issuance **(tokenization),** and financial inclusion. It enables global transfers in 2–5 seconds with very low fees and is supported by the Stellar Development Foundation.

Resources:

🔗 stellar.org — Official website
🔗 Stellar Overview Docs — Start here to understand the fundamentals

2. Why Use Stellar for Fintech?

- Built-in payment system
- Built-in asset issuance (no smart contract required)
- Low transaction fees
- Fast settlement

- Support for stablecoins and tokenized assets

Resources:

🔗 Stellar Use Cases — Real-world applications and inspiration

3. Core Building Blocks

- Accounts (Public & Secret Keys)
- XLM (native currency for fees and minimum balance)
- Transactions (must be signed before submission)
- Assets (custom tokens issued by an account)
- Trustlines (required before receiving custom assets)
- Soroban Smart Contracts (advanced logic layer)

Resources:

🔗 Accounts & Keys
🔗 Assets & Trustlines
🔗 Soroban Smart Contracts
🔗 Stellar Laboratory — Browser-based tool to explore accounts and build transactions hands-on

4. Development Tools

- JavaScript: npm install stellar-sdk
- Python: pip install stellar-sdk
- Use Stellar Testnet during the hackathon

Resources:

🔗 JavaScript SDK Docs
🔗 Python SDK Docs
🔗 Stellar Testnet Friendbot — Fund a testnet account instantly
🔗 Testnet Explorer (Stellar Expert) — See your testnet transactions live

5. What You Must Prepare Before the Hackathon

- Install Node.js or Python
- Install Stellar SDK
- Generate and fund a Testnet account

- Successfully send one test payment
- Join the community Discord

Resources:

🔗 Node.js Download
🔗 Python Download
🔗 Quickstart Tutorial (JavaScript)
🔗 Stellar Developer Discord — Ask questions, get help from the community

6. Suggested Project Ideas

- Cross-border payment prototype
- Stablecoin or token issuance demo
- Merchant payment solution
- Savings group or microfinance tool

Resources:

🔗 Stellar Build Tutorials — Step-by-step guides for common project types
🔗 Example Projects on GitHub — Browse official Stellar repos for inspiration

7. Judging Expectations

- Use Stellar Testnet
- Include at least one real transaction
- Demonstrate understanding of payments or assets
- Present a working demo with explanation

Resources:

🔗 Mainnet vs Testnet Explained — Know the difference before demo day
🔗 Stellar Expert (Testnet) — Use this to show live transactions during your demo

8. Building a Cross-Border Payment API Guide

This section is specifically for students building a cross-border payment prototype. It walks you through the key APIs and code patterns you will need, from creating accounts to sending a multi-currency path payment.

8.1 The Horizon API ; Your Gateway to Stellar

Horizon is Stellar's HTTP REST API. Every interaction with the network — checking balances, submitting transactions, finding payment paths — goes through Horizon. You don't call it directly in most cases; the SDK wraps it for you. But it helps to know what is happening underneath.

Testnet base URL: https://horizon-testnet.stellar.org
Mainnet base URL: https://horizon.stellar.org

Key Horizon endpoints you will use:

- **GET /accounts/{id}** — fetch account balance and sequence number
- **GET /accounts/{id}/payments** — list all past payments for an account
- **GET /paths/strict-receive** — find conversion paths between two currencies
- **POST /transactions** — submit a signed transaction to the network

🔗 Horizon API Reference

🔗 Horizon Endpoints Explorer (Stellar Lab)

8.2 Step-by-Step: A Cross-Border Payment in JavaScript

The scenario: a sender in Tanzania wants to send TZS (Tanzanian shilling), and a recipient in the US receives USDC. Stellar's path payment automatically handles the currency conversion on-chain.

Step 1 — Set up and connect to Horizon (Testnet)

```
const StellarSdk = require('stellar-sdk');

// Point to Testnet during development
const server = new StellarSdk.Horizon.Server(
  'https://horizon-testnet.stellar.org'
);
const networkPassphrase = StellarSdk.Networks.TESTNET;
```

Step 2 — Create or load sender and receiver accounts

```
// Generate a new keypair (do this once, save the keys securely)
const senderKeypair = StellarSdk.Keypair.random();
console.log('Public Key:', senderKeypair.publicKey());
console.log('Secret Key:', senderKeypair.secret());

// Fund on Testnet using Friendbot
// Visit: https://friendbot.stellar.org?addr=<YOUR_PUBLIC_KEY>
```

```
// Load an existing account
const senderAccount = await server.loadAccount(senderKeypair.publicKey());
```

Step 3 — Find a payment path (currency conversion route)

Before sending, ask Horizon if a conversion path exists between your source asset and the destination asset. This is what makes cross-border payments possible without a central exchange.

```
// Find paths to send USDC to the recipient
const USDC_ISSUER =
'GA5ZSEJYB37JRC5AVCIA5MOP4RHTM335X2KGX3IHOJAPP5RE34K4KZVN';
const usdcAsset = new StellarSdk.Asset('USDC', USDC_ISSUER);

const paths = await server.strictReceivePaths(
  senderKeypair.publicKey(),  // source account
  usdcAsset,                  // destination asset
  '10'                        // recipient must receive exactly 10 USDC
).call();

console.log('Available paths:', paths.records);
// Each record shows you the conversion rate and intermediate assets
```

Step 4 — Build and submit a Path Payment transaction

A Path Payment lets you send one asset and have the recipient receive a different asset — Stellar does the conversion automatically using its built-in DEX.

```
const fee = await server.fetchBaseFee();

const transaction = new StellarSdk.TransactionBuilder(senderAccount, {
  fee,
  networkPassphrase
})
  .addOperation(StellarSdk.Operation.pathPaymentStrictReceive({
    sendAsset: StellarSdk.Asset.native(), // Sender pays with XLM
    sendMax: '50',          // Max XLM willing to spend
    destination: receiverPublicKey,
    destAsset: usdcAsset,   // Recipient gets USDC
```

```
    destAmount: '10',       // Recipient gets exactly 10 USDC
    path: [],               // Leave empty to let Stellar auto-route
  }))
  .setTimeout(30)
  .build();

// Sign the transaction with the sender's secret key
transaction.sign(senderKeypair);

// Submit to the network
const result = await server.submitTransaction(transaction);
console.log('Success! Tx hash:', result.hash);
```

Step 5 — Check transaction status and payment history

```
// Look up a transaction by hash
const tx = await server.transactions()
  .transaction(result.hash)
  .call();
console.log('Transaction:', tx);

// List all payments for an account
const payments = await server.payments()
  .forAccount(receiverPublicKey)
  .limit(10)
  .order('desc')
  .call();
console.log('Recent payments:', payments.records);
```

> 💡 **Key Concept:** *pathPaymentStrictReceive guarantees the recipient gets the exact amount specified. pathPaymentStrictSend guarantees the sender spends the exact amount specified. For remittances, strict receive is usually better — the recipient knows exactly what they'll get.*

🔗 Full JS SDK payment examples
🔗 Path Payments explained
🔗 Build a payment app tutorial (full walkthrough)

8.3 SEP Standards — How Real Fintech Apps Connect

SEP stands for Stellar Ecosystem Proposal. These are open standards that define how wallets, anchors, and applications talk to each other. If you want your app to connect to real money (not just testnet tokens), you will need to interact with SEP-compliant anchor APIs.

- **SEP-10** — Authentication. How your app identifies a user to an anchor (like logging in).
- **SEP-24** — Deposit and withdrawal. Your user interacts with an anchor's UI inside your app to move fiat in/out.
- **SEP-31** — Direct cross-border payments. Two anchors talk to each other server-to-server; the customer never sees it. Used for bank-to-bank remittances.
- **SEP-38** — Quote protocol. Get a firm exchange rate quote before committing to a transaction.

For the hackathon, you do not need to build a full SEP server. But you should understand that real Stellar apps use these standards, and your demo can simulate SEP-24 or SEP-31 flows even without connecting to a live anchor.

🔗 SEP-24: Deposit & Withdrawal guide
🔗 SEP-31: Cross-border payments guide
🔗 All SEP specifications

9. Understanding Anchors

Anchors are the bridge between the Stellar blockchain and real-world money. They are regulated financial companies that hold fiat currency in reserve and issue equivalent tokens on the Stellar network. When someone wants to move real USD, KES, TZS, or EUR through Stellar, they go through an anchor.

Think of an anchor as a local bank or money transfer operator that has agreed to follow Stellar's open standards. They handle the compliance (KYC, AML) and the fiat side; your application handles the Stellar side.

9.1 How Anchors Work in a Cross-Border Payment

Here is the flow for a real cross-border payment using anchors:

```
Sender (Tanzania)        Stellar Network          Recipient (USA)
____________________________________________________________________________
_____________________

1. Deposits TZS cash   →  Sending Anchor issues
   at anchor agent        TZS token on Stellar
                     →  Path Payment converts
                        TZS → XLM → USDC
                     →  Receiving Anchor        → 2. Recipient
                        receives USDC               withdraws USD
                                                 from bank/agent
```

9.2 Key Anchors to Know

These are real anchor organizations building on Stellar. During your hackathon you will use the Testnet, but these are the players your demo could interact with in production:

Anchor	Region / Focus	Assets	Role / Why It Matters
MoneyGram	Global (180+ countries)	USDC ↔ Cash	Lets users convert physical cash to/from USDC at agent locations — no bank account needed. Critical for last-mile access.
Circle (USDC)	Global	USDC	Issues the USDC stablecoin on Stellar. The most widely used dollar token on the network.

Anchor	Region / Focus	Assets	Role / Why It Matters
Yellow Card	Africa (20 countries)	USDC, local currencies	Largest licensed stablecoin on/off-ramp in Africa. Covers Nigeria, Kenya, Tanzania, Ghana and more.
Tempo	Europe / Asia	EURC (Euro)	French-licensed money transfer provider. Issues Euro stablecoin on Stellar for EU remittances.
Airtm	Latin America	USDC, local fiat	Digital dollar wallet for emerging markets. Enables enterprise payouts to freelancers and gig workers.
Stronghold	USA	USD tokens	First venture-backed USD anchor on Stellar. Issues tokenized US dollars for enterprise settlement.
Bitso	Mexico / LATAM	MXN, USDC	Used for USD-to-MXN remittance corridors. Powers flows from US Hispanics sending money to Mexico.

🔗 Stellar Anchor Directory — Browse all 69 anchors by country and currency
🔗 MoneyGram Ramps — Integration guide for cash on/off-ramps in 180+ countries

9.3 Africa-Specific Context

For students building solutions relevant to East Africa (Tanzania, Kenya, Uganda), these are the most important players:

- **Yellow Card** — Active in Tanzania, Kenya, Uganda, Nigeria, Ghana. Supports USDC on Stellar. Best option for a Tanzanian use case.
- **MoneyGram + Honeycoin** — USDC cash-out at MoneyGram locations is already live in Tanzania, Kenya, Uganda, Nigeria, Ghana, and the UK.
- **Fonbnk** — Converts mobile airtime credit into stablecoins. Useful for users without bank accounts across Sub-Saharan Africa.
- **Ejara** — Wallet serving Francophone Africa, integrated with MoneyGram Ramps.

🔗 Yellow Card — Africa's largest stablecoin ramp
🔗 Fonbnk — Airtime to stablecoin conversion

9.4 How to Discover an Anchor's API (TOML file)

Every Stellar anchor publishes a machine-readable configuration file called stellar.toml. This file tells you what assets they support, what SEP standards they implement, and what API endpoints to call.

To find an anchor's TOML file, look it up at:

```
https://<anchor-domain>/.well-known/stellar.toml

// Example: fetch MoneyGram's anchor info
fetch('https://extstellar.moneygram.com/.well-known/stellar.toml')
  .then(res => res.text())
  .then(toml => console.log(toml));

// The TOML file will contain:
// - TRANSFER_SERVER_SEP0024 = 'https://...' (deposit/withdraw endpoint)
// - SIGNING_KEY = 'G...' (anchor's public key)
// - Supported currencies and conditions
```

🔗 How to read a stellar.toml file

Bonus Resources

🔗 Stellar Quest — Gamified challenges to learn Stellar by doing (highly recommended!)
🔗 Stellar YouTube Channel — Video walkthroughs and developer talks
🔗 Stellar Blog — Stay current on ecosystem developments
🔗 2024 Stellar Year in Review — Latest ecosystem updates

> 💡 **Pro Tip:** *The most important links to check right away are Stellar Laboratory, Friendbot, and Stellar Quest — those three alone can get a student from zero to their first transaction in under an hour. For cross-border builds, also read the Anchor Directory to understand who your app would connect to in the real world.*

Stellar